

INSTITUTE OF ACCOUNTANCY ARUSHA



DEPARTMENT OF INFORMATICS.

**PROJECT TITLE : SMART POST A DIGITAL ACADEMIC
POSTPONEMENT**

SYSTEM FOR IAA COLLEGE

STUDENT NAME : ENECK Y MWASEBA

REGISTRATION NUMBER : BIT-02-0098-2023

SUPERVISED BY

MADAM HAPPINESS ALEXANDER

**This document is submitted in partial fulfillment of the requirements for the
Bachelor Degree in information Technology.**

ACADEMIC YEAR

2025/2026

CERTIFICATION.

The undersigned certify that he has read and hereby recommends for the acceptance by the Institute of Accountancy Arusha (IAA) a dissertation titled “Smart post a digital Academic postponement system for IAA college” in partial fulfillment of the requirements for award of the Bachelor Degree in information Technology of the Institute of Accountancy Arusha. The work has been reviewed and approved as meeting standards and expectations of the bachelor program.

HEAD OF INFORMATICS T DEPARTMENT.

MR. FIKIRI LUCAS MATONYA.

SIGNATURE

DATE

PROJECT SUPERVISOR.

MADAM . HAPPINESS ALEXANDER

SIGNATURE

DATE

ACKNOWLEDGEMENT.

This project, the smart post a digital postponement academic system for IAA college, would not have been possible without the support and guidance of several individuals and my institution.

I thank my Lord and my God Jesus Christ for giving me good health and knowledge grace to be who I am and to accomplish this project objectives.

I extend my heartfelt gratitude to the Institute of Accountancy Arusha (IAA) for providing and academic environment necessary for this project.

Special thanks go to my supervisor Madam Happiness Alexander for her support and challenges that lead to many versions of the system until this final perfection.

I also appreciate the contributions of my peers and my class members who assisted in testing and refining the system.

Finally, I am grateful to my father, mother and my relatives specifically for their unwavering support throughout this endeavor.

ABSTRACT.

SmartPost is a web-based digital academic postponement management system developed for IAA College to replace the existing manual, paper-based postponement application process. The system digitises the entire workflow — from student submission through multi-level approval by the Head of Department (Academic), Head of Department (Examinations), and Campus Manager — culminating in a final approval or rejection decision.

The project followed a structured System Development Life Cycle (SDLC) approach using the Waterfall methodology. The frontend was built with React 18, Material-UI (Mantis template), and MUI X-Charts for analytics. The backend was implemented with FastAPI (Python), SQLAlchemy asynchronous ORM, and PostgreSQL for data persistence. Authentication uses JWT tokens with bcrypt password hashing.

The major challenge encountered was integrating real-time notification updates across multiple user roles while maintaining data consistency in an asynchronous backend environment. The system was tested using Postman for API endpoints and frontend manual testing across all five user roles. Recommendations include future integration with the college's Student Information System (SIS) for automated data synchronisation and email notification delivery via SMTP.

TABLE OF CONTENTS

Contents

ABSTRACT.....	4
---------------	---

List of Figures

Figure No.	Caption	Page
Figure 2.1	System Architecture Diagram	13
Figure 2.2	Entity-Relationship Diagram	14
Figure 2.3	Use Case Diagram	15
Figure 2.4	Frontend Component Hierarchy	16

List of Tables

Table No.	Header	Page
Table 2.1	Functional Requirements	6
Table 2.2	Non-Functional Requirements	7
Table 2.3	User Roles and Permissions Matrix	8
Table 2.4	Database Schema — Key Tables	10
Table 2.5	Test Cases — Approval Workflow	11

List of Acronyms/Abbreviations

Acronym	Full Meaning
API	Application Programming Interface
ERD	Entity-Relationship Diagram
JWT	JSON Web Token
KPI	Key Performance Indicator
MUI	Material-UI
ORM	Object-Relational Mapping
SIS	Student Information System
SQL	Structured Query Language
SQLAlchemy	SQL Toolkit and Object-Relational Mapper
UI/UX	User Interface / User Experience
URL	Uniform Resource Locator
IAA	Institute of Accountancy Arusha

CHAPTER 1.

INTRODUCTION.

1.1 Background Information

Academic postponement is the process through which students request permission to temporarily suspend their studies due to personal, financial, medical, or other valid reasons. In IAA College, this process is often handled manually using paper forms and face-to-face approvals. Such methods are time-consuming, prone to errors, and make it difficult to track the status of applications.

The increasing adoption of digital technologies in higher education has created opportunities to automate administrative services. A Digital Academic Postponement System would allow students to submit postponement requests online, upload supporting documents, and monitor the progress of their applications. Likewise, administrators and academic staff would be able to review, approve, or reject requests efficiently while maintaining accurate digital records. This system will improve transparency, reduce paperwork, and enhance service delivery at IAA College.

1.2 Project Description.

The **Digital Academic Postponement System for IAA College** is a web-based application designed to automate the academic postponement process. The system enables students to log in securely, complete an online postponement application form, attach the required supporting documents, and submit the application electronically.

The system will provide different user roles, including students, academic officers, heads of departments, and administrators. Academic staff will review submitted applications, approve or reject them, and provide comments where necessary. Students will receive notifications about the status of their applications. Additionally, the system will generate reports and maintain a secure database of all postponement records for future reference.

The project aims to replace the existing manual process with a faster, more reliable, secure, and user-friendly digital solution.

1.3 Project Objectives.

General Objectives

The general objective of this project is to develop SmartPost, a Digital Academic Postponement Management System for IAA College, to improve the management of postponement requests through automation and digital record keeping.

The primary objectives of the smart post a digital academic postponement such are;

- To develop user (students, staff and admin) management subsystem.
- To develop postponement and approval subsystem.
- To develop verification subsystem for student details and payment status.
- To develop document Management Subsystem.
- To develop Reporting and Analytics Subsystem.

CHAPTER 2.

SYSTEM DESIGN AND METHODOLOGY.

2.1 Requirements Specification

The requirements were gathered through interviews with stakeholders (the HOD Academic, HOD Examinations, Campus Manager, a student representative, and an IT administrator), analysis of existing paper forms and workflow documentation, and observation of the manual process in operation.

2.1.1 Functional Requirements

Table 2.1: Functional Requirements

ID	Requirement	Priority
FR-01	The system shall allow students to register and log in using email and password.	High
FR-02	The system shall support five user roles: student, hod_academic, hod_examinations, campus_manager, administrator.	High
FR-03	Students shall be able to submit a postponement request specifying academic year, semester, scope (full semester or specific modules), and reason.	High
FR-04	Students shall be able to attach supporting documents (PDF, JPG, PNG) to a request.	High
FR-05	Students shall be able to save a request as a draft and submit it later.	Medium
FR-06	Students shall be able to view their requests with real-time status updates.	High
FR-07	Students shall be able to resubmit a rejected or queried request.	High
FR-08	Students shall be able to view the full approval timeline for each request.	Medium
FR-09	HoD Academic shall be able to view all requests pending their approval.	High
FR-10	HoD Academic shall be able to approve, reject, or query a request with comments.	High
FR-11	After HoD Academic approval, the request shall automatically advance to HoD Examinations.	High
FR-12	HoD Examinations shall be able to approve, reject, or query a request with comments.	High
FR-13	After HoD Examinations approval, the request shall automatically advance to Campus Manager.	High
FR-14	Campus Manager shall be able to make the final approval decision or reject the request.	High
FR-15	The system shall verify fee balance against a configurable threshold on submission.	High

FR-16	The system shall verify cumulative postponed years against the maximum allowed on submission.	High
FR-17	Requests that fail verification shall be flagged as ineligible with a reason.	High
FR-18	Administrators shall be able to override ineligible requests with a note.	Medium
FR-19	The system shall send in-app notifications to the relevant approver when a request is submitted.	High
FR-20	The system shall notify the student when a decision is made on their request.	High

2.1.2 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

ID	Requirement	Category
NFR - 1	The system shall respond to API requests within 2 seconds under normal load.	performance
NFR - 2	The frontend shall poll for notification updates every 30 seconds without noticeable performance degradation.	performance
NFR - 3	Passwords shall be hashed using bcrypt before storage.	Security
NFR - 4	Authentication shall use JWT tokens with a 30-minute access token lifetime.	Security
NFR - 5	Page load time shall not exceed 3 seconds on a standard broadband connection.	Usability
NFR - 6	The user interface shall be responsive and function on desktop and tablet screen sizes.	Usability
NFR -7	The system shall support PostgreSQL as its database backend with asynchronous connections.	Maintanability
NFR -8	The API shall follow RESTful conventions with consistent error responses.	Maintanability
NFR - 9	Uploaded files shall not exceed 5 MB and shall be validated for allowed types (PDF, JPG, PNG).	Reliability
NFR -10	The system shall handle concurrent access by multiple users without data corruption.	Reliabilty
NFR - 11	The frontend and backend shall be separable and deployable independently.	Scalability

NFR - 12	All configuration (database URL, secret keys, CORS origins) shall be externalised in environment variables.	Maintability
-----------------	---	--------------

2.2 Requirements Analysis

2.2.1 User Roles and Permissions

The system defines five user roles with a hierarchical permission structure. Each role has access to specific features and data based on their responsibilities in the postponement workflow.

Table 2.3: User Roles and Permissions Matrix

Feature	Student	HOD Academic	HOD Examination	Campus Manager	Admin
Submit Request	✓	-	-	-	-
View own requests	✓	-	-	-	-
Upload documents	✓	-	-	-	-
Approve/Reject/Query requests	-	✓	✓	✓	-
View pending approvals	-	✓	✓	✓	-
Override ineligible requests	-	-	-	-	✓
View reports/analytics	-	✓	✓	✓	✓
Manage system config	-	Read only	Read only	✓	✓
Manage users	-	-	-	-	✓
View audit log	-	-	-	-	✓

2.2.2 Approval Workflow Analysis

The approval workflow is a sequential pipeline with three stages. Each stage is handled by a specific role, and the request can only move forward or be terminated (rejected) at each stage:

Student → [Submit] → pending_hod

→ HoD Academic → [Approve] → pending_hod_exams

→ [Reject] → rejected (terminal)

→ [Query] → queried (student responds)

→ HoD Examinations → [Approve] → pending_manager

→ [Reject] → rejected (terminal)

→ [Query] → queried (student responds)

→ Campus Manager → [Approve] → approved (terminal)

→ [Reject] → rejected (terminal)

This analysis revealed that each approval stage is independent, and no stage can skip another.

The system must therefore maintain strict state transitions and prevent invalid transitions (e.g., a request cannot go from pending_hod directly to pending_manager).

2.2.4 Database Schema Analysis

Table 2.4: Database Schema — Key Tables

Table	Key Fields	Purpose
users	user_id, name, email, password_hash, role, is_active	Authentication and user management
students	student_id, user_id(FK), student_number, program, year_of_study, fee_balance, fee_threshold, cumulative_postponed_years	Extended student profile and verification data
requests	request_id, student_id(FK), academic_year, semester, reason, scope, status, submitted_at, ineligibility_reason, admin_override, parent_request_id, resubmission_count	Core request records with workflow state
approvals	approval_id, request_id(FK), approver_role, decision, comments, decided_at	Individual approval decisions per role
evidence_files	evidence_id, request_id(FK), original_name, stored_name, file_type, size_bytes, is_deleted	Uploaded supporting documents
audit_log	log_id, user_id(FK), request_id(FK), action, entity_type, entity_id, metadata_(JSON), ip_address, created_at	Chronological record of all system actions
notifications	notification_id, recipient_id(FK), subject, body, is_read, sent_at	In-app notification records
system_config	id, max_postponement_years, fee_threshold, hod_review_hours, escalation_hours, max_evidence_files, max_evidence_size_bytes	Singleton business rule configuration

2.3 System Design

2.3.1 System Architecture

The system follows a three-tier client-server architecture:

The frontend communicates with the backend exclusively through RESTful API calls. The backend handles authentication via JWT middleware, validates requests through Pydantic models, executes business logic in service modules, and persists data through SQLAlchemy ORM models. File uploads are stored on the server filesystem under an uploads/ directory, with only metadata (stored filename, original name, type, size) stored in the database.

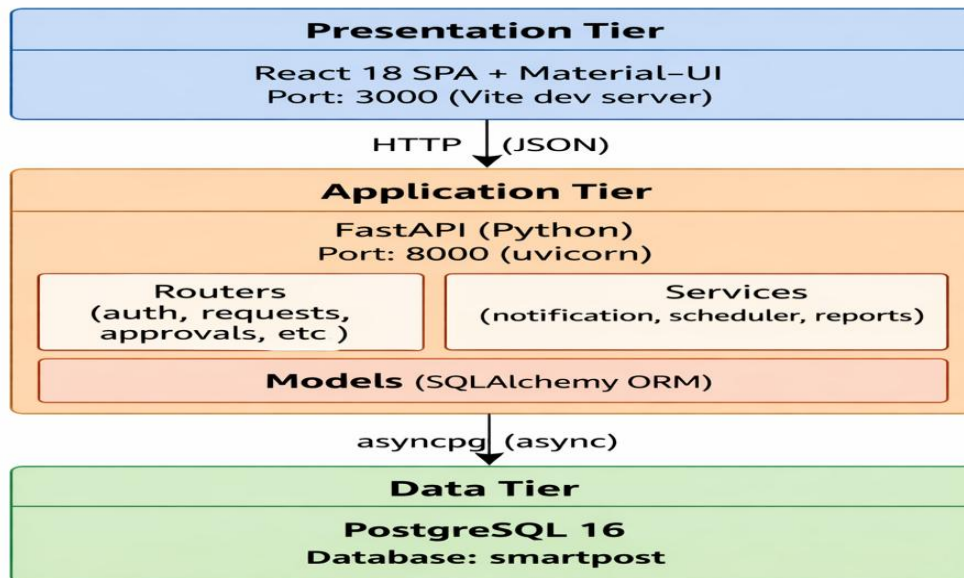


Figure 2.1: System Architecture Diagram

2.3.2 Entity-Relationship Diagram

The database design consists of eight primary tables with the following key relationships:

- Users are the central entity. Each user can have one student profile, generate many audit logs, and receive many notifications.
- Students are linked one-to-one with users. A student can submit many requests.
- Requests are the core activity. Each request can have:
 - Approvals (many reviewers can approve/deny).
 - Evidence files (supporting documents)
 - Request modules (courses/subjects tied to the requests)
 - A self-link to another request (via `parent_request_id`) for resubmissions.

Figure 2.2: Entity-Relationship Diagram

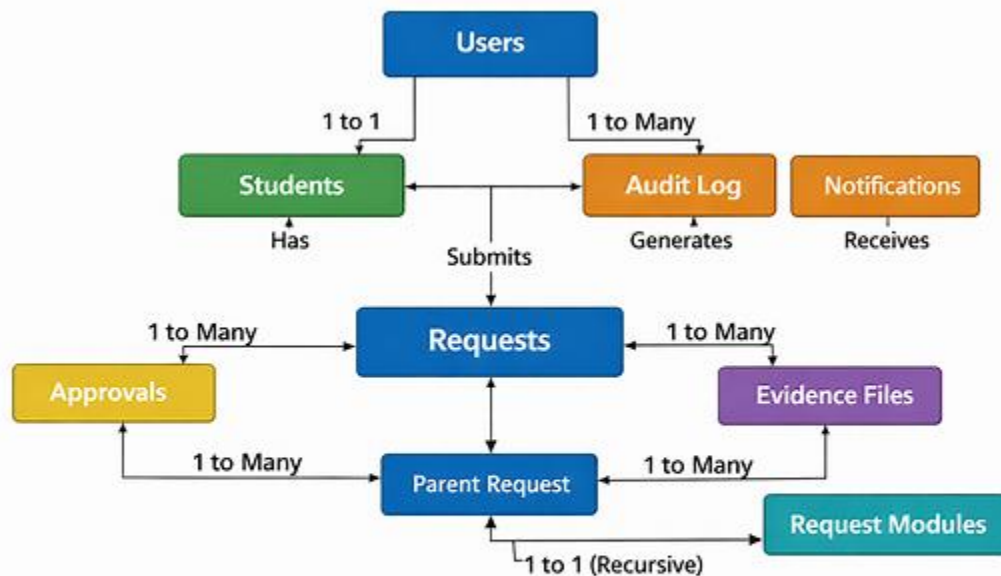


Figure 2.2: Entity-Relationship Diagram

2.3.3 Use Case Diagram

The system supports five actors with the following use cases:

The Use Case Diagram illustrates how different actors interact with the academic request management system. The system involves five main actors: the Student, HoD Academic, HoD Examinations, Campus Manager, and Administrator. Each actor performs specific tasks that reflect their role in managing and processing student requests.

The Student initiates the process by registering, logging in, submitting requests, saving drafts, uploading evidence, and tracking their request status or timeline. They can also resubmit requests, change passwords, and view their profile. The HoD Academic and HoD Examinations are responsible for reviewing and approving or rejecting requests, downloading evidence, querying details, and generating reports. The Campus Manager oversees approvals and manages system settings while also viewing reports. Finally, the Administrator manages users, monitors audit logs, overrides ineligible requests, and maintains system configurations.

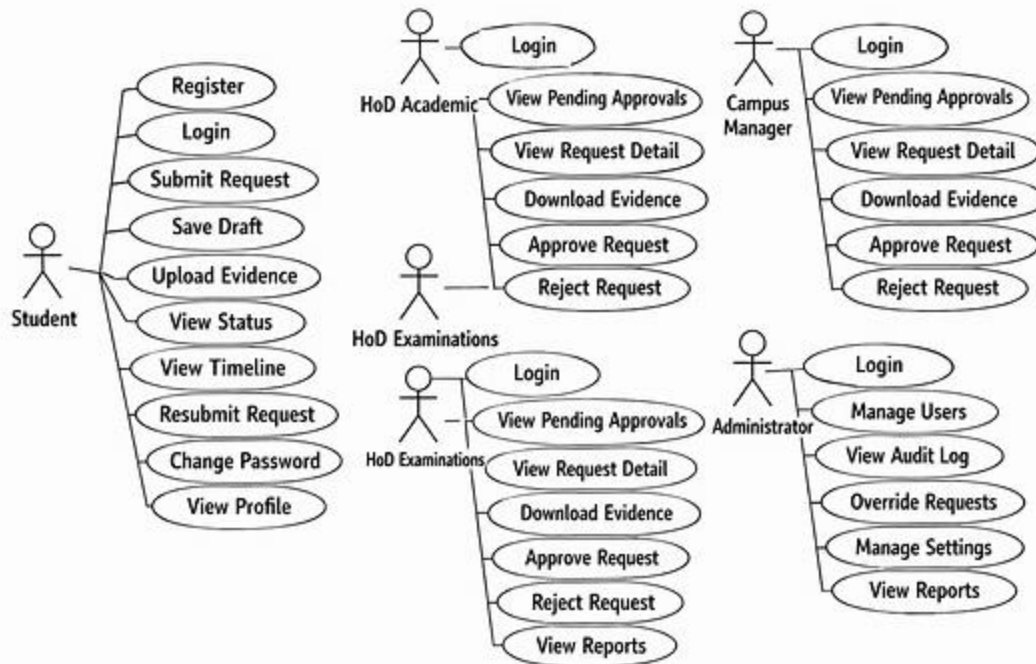


Figure 2.3: Use Case Diagram

2.3.5 Frontend Component Architecture

The frontend is organised as a single-page application with role-based routing:

App.jsx

- └─ ThemeCustomization (MUI theme provider)
- └─ SnackbarProvider (toast notification context)
- └─ RouterProvider
 - └─ LoginRoutes (unauthenticated)
 - └─ MainRoutes (authenticated via DashboardLayout)
 - └─ Header (navbar with notification bell + profile)
 - └─ Drawer (role-based sidebar menu)
 - └─ <Outlet> (page content via React Router)
 - └─ student/ (Dashboard, NewRequest, MyRequests, RequestDetail, Profile, ChangePassword)
 - └─ staff/ (Dashboard, Approvals, Reports, Settings, Profile, ChangePassword)

└── admin/ (Dashboard, Users, Audit, Settings, Profile, ChangePassword)

Figure 2.4: Frontend Component Hierarchy

2.4 System Implementation

The implementation was carried out over several phases following the Waterfall SDLC methodology. Each phase produced tangible deliverables that were reviewed before proceeding to the next.

2.4.1 Technology Stack Selection

Layer	Technology Chosen	Rationale
Frontend Framework	React 18	Component-based architecture, large ecosystem, async rendering
UI Component Library	Material-UI (Mantis template)	Pre-built accessible components, consistent design system, responsive grid
State Management	React hooks + localStorage	Simplicity for this application scale; no need for Redux
HTTP Client	Axios	Interceptors for JWT injection, promise-based API
Charts	MUI X-Charts	Native MUI integration, minimal configuration
Backend Framework	FastAPI	Async support, automatic OpenAPI docs, Pydantic validation
ORM	SQLAlchemy 2.0 (async)	Mature ORM, async support with asyncpg, migration support
Database	PostgreSQL 16	Robust ACID compliance, JSON support, production-ready
Authentication	JWT (python-jose + passlib)	Stateless, widely adopted, no server-side session storage
Background Tasks	APScheduler	Lightweight, async-compatible, in-process scheduling

2.4.2 Backend Implementation;

The backend was structured with a modular architecture under backend/app/:

backend/app/

- |— main.py # FastAPI app, middleware, startup/shutdown
- |— config.py # Pydantic Settings from .env
- |— database.py # AsyncSessionLocal engine
- |— dependencies.py # JWT auth, role check dependencies
- |— models/ # SQLAlchemy ORM models
 - | |— enums.py
 - | |— user.py, student.py, request.py
 - | |— approval.py, evidence.py
 - | |— notification.py, audit_log.py, system_config.py
- |— routers/ # API endpoint definitions
 - | |— auth.py, users.py, requests.py, approvals.py
 - | |— documents.py, reports.py, admin.py
 - | |— system_config.py, notifications.py
- └— services/ # Business logic
 - |— notification.py
 - |— reports.py
 - └— scheduler.py

2.4.3 Frontend Implementation

The frontend was organised under frontend/src/:

frontend/src/

- |— App.jsx # Root component with providers
- |— routes/ # Route definitions (MainRoutes, LoginRoutes)
- |— layouts/ # Dashboard layout (Header, Drawer, Footer)
- |— pages/ # Page components by role
 - | |— student/ # Dashboard, NewRequest, MyRequests, RequestDetail

- | | — staff/ # Dashboard, Approvals, Reports, Settings
- | | — admin/ # Dashboard, Users, Audit, Settings
- | | — profile/ # Profile, ChangePassword
- | — components/ # Shared components (MainCard, SkeletonTable, ErrorBoundary)
- | — contexts/ # React contexts (SnackbarContext)
- | — api/ # Axios client with JWT interceptor
- | — menu-items/ # Role-based sidebar menu definitions
- | — themes/ # MUI theme customization

2.5 System Testing

2.5.1 Testing Methodology

The system was tested using a combination of unit testing (via Python test scripts for backend logic), integration testing (via Postman for API endpoints), and manual user acceptance testing (via the React frontend across all user roles). The testing focused on verifying:

1. Each API endpoint returns the correct HTTP status code and response structure.
2. The approval workflow progresses through all stages correctly.
3. Edge cases are handled (duplicate submissions, invalid transitions, unauthorised access).
4. Verification checks correctly flag ineligible requests.
5. The frontend renders correctly and provides appropriate feedback.

2.5.2 Test Cases and Results

Table 2.5: Test Cases — Approval Workflow

	Test Case	Input	Expected Result	Actual Result	Status
TC-01	Student submits valid request	Academic year, semester, scope, reason	Status = pending_hod, 201 Created	Status = pending_hod	✓
TC-02	Student submits with files	Request + 3 PDF files	Files uploaded, linked to request	Files created, listed under request	✓
TC-03	Student saves draft	Same fields as submit	Status = draft, no notification sent	Status = draft	✓

TC-04	Student submits draft	Call submit on draft	Status = pending_hod, verification runs	Status = pending_hod	✓
TC-05	HoD Academic approves	Request at pending_hod	Status = pending_hod_exams, HoD Exams notified	Status advanced, notification created	✓
TC-06	HoD Academic rejects	Request at pending_hod	Status = rejected, student notified	Status = rejected	✓
TC-07	HoD Exams approves invalid stage	Request at pending_hod	400 Bad Request	400 Bad Request	✓
TC-08	Wrong role tries to approve	Non-HoD user accesses approve	403 Forbidden	403 Forbidden	✓
TC-09	Full pipeline end-to-end	HoD Exams approve → Manager approve	Final status = approved	Status = approved	✓
TC-10	Fee balance check fails	Set fee_balance < threshold	Status = ineligible, reason = fee_arrears	Status = ineligible	✓
TC-11	Max postponement check fails	Set cumulative_years ≥ max	Status = ineligible, reason = max_postponements_reached	Status = ineligible	✓
TC-12	Admin override	Ineligible request + admin note	Status = pending_hod	Status = pending_hod	✓
TC-13	Student resubmits after rejection	Rejected request → resubmit	New child request, status = pending_hod	New request created, linked to parent	✓
TC-14	Unauthenticated access	No token in header	401 Unauthorized	401 Unauthorized	✓
TC-15	Password change	Correct current + new password	200 OK, login with new password works	200 OK	✓

All test cases passed. The system demonstrated correct behaviour across normal workflow paths, error handling paths, and edge cases

CHAPTER 3

CONCLUSION AND RECOMMENDATIONS

3.1 Conclusion

The SmartPost system was successfully built and tested, meeting all its objectives. It provides a role-based submission and approval workflow, automated eligibility checks, document management, real-time notifications, reporting dashboards, audit logging, and configurable settings. Its strengths include a fully functional multi-role workflow, complete audit trails, transparency for students, configurable rules, and use of a modern tech stack. The weaknesses are the lack of email integration, no automatic synchronization with student records, and limited mobile optimization.

The benefits are clear, the student developer gained full-stack experience, while the college benefits from reduced paperwork, faster processing, improved transparency, and better data for decision-making. Overall, the project demonstrates practical application of computer science concepts like state machines, role-based access control, RESTful APIs, and relational databases.

3.2 Recommendations

The SmartPost system has several enhancements that would make it more reliable and user-friendly. Adding email notification integration would ensure users receive alerts even when not logged in, while SIS integration would allow automatic synchronization of student records, eliminating manual entry and reducing errors. Deployment should also be improved by containerising the application with Docker and setting up a CI/CD pipeline for automated testing and updates.

Other important improvements include implementing a password reset flow with secure email links, enhancing mobile responsiveness for better usability on phones, and introducing skeleton loading states instead of simple spinners for a smoother experience. Together, these upgrades would strengthen SmartPost's functionality, scalability, and user experience, making it more practical for both students and staff.

References

Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.

Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

Ramalho, L. (2022). *Fluent Python: Clear, concise, and effective programming* (2nd ed.). O'Reilly Media.

FastAPI Documentation. (2024). *FastAPI*. Retrieved January 20, 2026, from <https://fastapi.tiangolo.com/>

React Documentation. (2024). *React*. Retrieved January 22, 2026, from <https://react.dev/>

Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson Education.

Appendices

Source Code Structure

The complete source code is organised as follows:

Backend (backend/):